

OPEN-SOURCE ALPHA

PARIMIT

A Non-Dispatchable Payment-Intent Boundary for AI Agents

Technical Design and Implementation Status Paper v0.1



Implementation-aligned research prototype

Parimit Contributors / September 2026

LOCAL DEMO ONLY - NO UPI, BANK, PSP, WALLET, BLOCKCHAIN, OR LIVE PAYMENT CONNECTION

Contents

This paper distinguishes implemented behavior from target invariants. Bold target labels in later tables are not claims about the alpha.

Abstract	3
1. The design question	3
2. Contribution and claim boundary	3
3. Goals and non-goals	4
4. System architecture	5
5. Proposal and lifecycle model	5
6. Evidence and integrity mechanisms	6
7. Capability surfaces	7
8. Relationship to 2026 agent-payment protocols	8
9. Threat model and residual risk	9
10. Verification evidence	10
11. Hardening roadmap	10
12. Design invariants for future releases	10
13. Conclusion	11
References	11
Appendix A - Reproducibility snapshot	13
Appendix B - Independent project statement	13

Abstract

AI agents can discover products, negotiate with software services, and assemble transactions. The dangerous step is not describing a payment; it is acquiring the authority to approve, sign, dispatch, or retry one. Parimit explores a narrow architectural response: place a deterministic, non-dispatchable governance boundary between an untrusted agent and any future payment executor.

In Parimit v0.1-alpha an agent can create, inspect, simulate, and cancel a structured payment proposal through REST or a deliberately narrow Model Context Protocol (MCP) surface. Fixed policy checks determine whether a proposal is denied or sent to a demo review surface. Distinct demo actor identifiers can record one or two decisions. The resulting receipt explicitly states that execution is not authorized and contains no payment credential, provider endpoint, or dispatch method. A local SQLite store records proposals, decisions, mock observations, and hash-linked audit events.

The prototype demonstrates capability minimization and the separation of proposal, internal decision, and external execution. It does **not** demonstrate secure human authentication, production authorization, complete database tamper detection, concurrency safety, external settlement, compliance, or protocol conformance. New v2 intents bind the initial policy decision and initial status into the proposal digest, while the integrity verifier cross-checks current state against valid decisions and the local event history. These checks detect isolated database edits; they do not make a locally controlled database tamper-proof. `IN_DOUBT` is enforced fail-closed for the mock observation stream: every later mock observation is rejected, and the alpha exposes no reconciliation mechanism.

1. The design question

Most payment protocols answer some version of: how can a payment be requested, authorized, presented, verified, or settled? An agent governance layer must answer a different question first:

What is the smallest useful capability an AI agent can receive without also receiving a path to move money?

Parimit's answer is a payment **proposal**, not a payment instruction. A proposal is structured enough to evaluate, review, hash, audit, and later map to another protocol. It is intentionally insufficient to debit an account, sign an authorization, obtain a payment credential, submit a checkout, invoke a facilitator, or call a rail.

This separation addresses three recurring risks:

- **Authority collapse.** A single agent or tool has enough permissions to propose, approve, and execute its own action.
- **Intent substitution.** A user reviews one payee, amount, or purpose while a later component acts on another.
- **Uncertain-outcome duplication.** A timeout or ambiguous response is treated as failure and retried, potentially causing more than one payment.

The alpha demonstrates an architectural boundary around these risks. It is not yet a complete security implementation.

2. Contribution and claim boundary

The most defensible description of this release is:

Parimit v0.1-alpha is an independently implemented, open-source, self-hosted reference prototype for a non-dispatchable payment-intent governance boundary for AI agents.

The architecture is rail-neutral because execution is outside the core and no rail adapter ships. The current proposal schema is narrower: it accepts only INR amounts represented as decimal strings in paise. Therefore "rail-neutral" describes the boundary, not current universal currency support.

2.1 What the alpha contributes

- A proposal-only REST and MCP capability surface.

- A deterministic policy example with per-transaction, daily aggregate, allowlist, blocklist, expiry, and one- or two-actor decision rules.
- A state model that ends at `AUTHORIZED_NO_DISPATCH` rather than a paid or settled state.
- A versioned v2 SHA-256 digest over immutable proposal fields, initial status, and selected policy-decision fields, with archival read compatibility for legacy v1 rows.
- HMAC-authenticated individual decision records and hash-linked local audit events.
- A mock outcome simulator that never contacts an external service.
- Executable tests and a heuristic source scanner for selected boundary violations.

2.2 What the alpha does not claim

Parimit is not NPCI-backed, UPI-enabled, UPI-compatible, bank-grade, PCI-compliant, regulator-approved, production-ready, formally verified, or a payment processor. It does not implement AP2, x402, ACP, UPI, Reserve Pay, or any provider SDK. Its HMAC records are not digital signatures or public proof of a person's identity. Its local audit chain is not immutable and can be rewritten by an attacker who controls the database and process. The project makes no "first," "only," or market-uniqueness claim.

3. Goals and non-goals

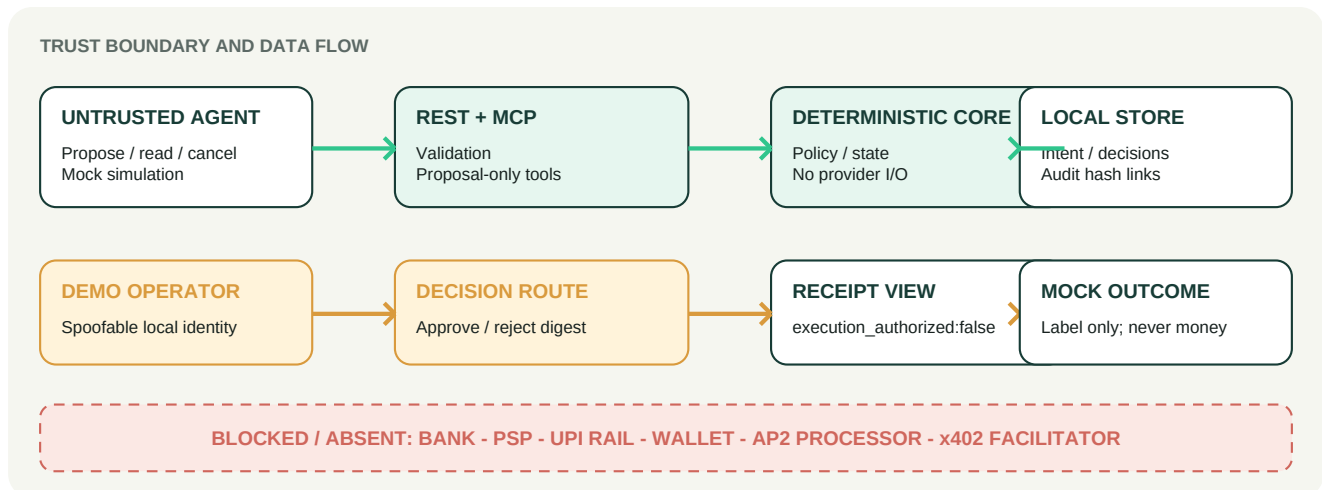
3.1 Design goals

- 1 **Capability minimization.** Agent-facing tools may propose and observe but expose no approval, credential, signing, execution, or retry capability.
- 2 **Deterministic evaluation.** Policy decisions occur in ordinary code rather than an LLM.
- 3 **Explicit review object.** Reviewers see a structured proposal whose immutable inputs and initial policy decision are bound to a versioned digest.
- 4 **Non-dispatchable evidence.** Internal approval evidence is structurally distinct from a provider instruction.
- 5 **Inspectable behavior.** A small implementation, tests, and safety scanner make the intended boundary reviewable.
- 6 **Protocol composability without authority leakage.** Future adapters may translate evidence at a separate boundary, but the core never signs or sends a live payment.

3.2 Non-goals for v0.1-alpha

- Identity proofing, passkeys, enterprise SSO, authorization servers, or tenant isolation.
- Real payment credentials, account linking, holds, debits, transfers, settlement, refunds, disputes, or reconciliation.
- A general fraud or risk engine.
- Strong concurrency across multiple processes.
- Tamper-proof storage, public verifiability, non-repudiation, or external audit anchoring.
- Compatibility certification for any commerce or payment protocol.

4. System architecture



The default build has three conceptual zones.

Untrusted proposal plane. An agent calls REST or MCP to create, read, simulate, or cancel a proposal. Inputs are validated and passed to deterministic policy code. The MCP registry contains no tool for approval, authorization, signing, sending, paying, executing, or retrying.

Demo decision plane. A browser dashboard calls a separate HTTP route to record approve or reject decisions against an exact proposal digest. In this release, caller identity and role arrive in spoofable headers. The application checks role strings, prevents a requester identifier from approving its own proposal, and requires distinct actor identifier strings for two-person review. These controls demonstrate workflow semantics; they are not authentication.

External execution plane. This zone is intentionally absent. No bank, PSP, UPI rail, wallet, AP2 credential provider, ACP merchant completion endpoint, or x402 facilitator is connected. `AUTHORIZED_NO_DISPATCH` means that the local demonstration recorded enough decisions; it does not mean funds are authorized on a rail.

4.1 Runtime and persistence

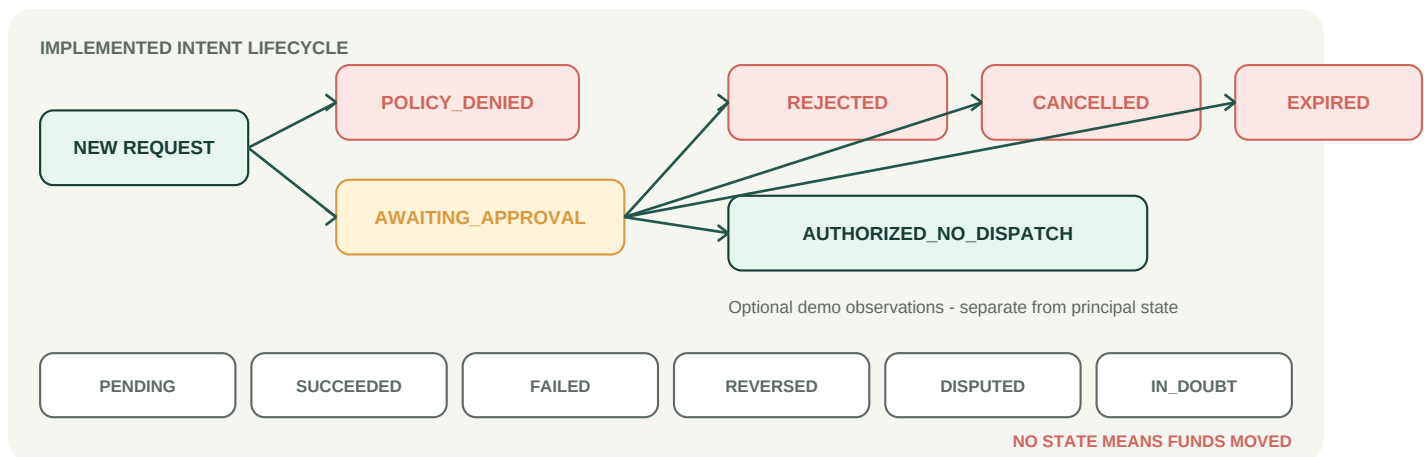
The reference implementation uses Node.js 24, TypeScript, built-in `node:sqlite`, and `node:crypto`, with no third-party runtime packages. Four SQLite tables store intents, approvals, observations, and audit events. The local HTTP process serves REST and static dashboard assets. The MCP process uses newline-delimited JSON-RPC over standard input and output. Both call the same application service, although separately started processes do not constitute a coordinated distributed transaction system.

4.2 Trust assumptions

Agent prompts, tool arguments, imported content, browser input, and network clients are untrusted. Demo actor identifiers are untrusted assertions. The host, local administrator, process memory, receipt secret, and database are not considered secure against a privileged attacker. Real providers are outside the system boundary.

5. Proposal and lifecycle model

An accepted request contains an agent-scoped idempotency key, requesting agent identifier, optional on-behalf-of label, positive INR amount in integer paise, opaque payee reference, purpose, and expiry. The implementation rejects floating-point money and identifiers that resemble a conventional UPI address, but this is only a syntactic guard and not a general detector of sensitive payment data.



The persisted lifecycle begins after validation and policy evaluation:

- A denied request is stored as `POLICY_DENIED`.
- An allowed request is stored as `AWAITING_APPROVAL`.
- `AWAITING_APPROVAL` may become `REJECTED`, `CANCELLED`, `EXPIRED`, or `AUTHORIZED_NO_DISPATCH`.
- Mock observation labels - `UNKNOWN`, `PENDING`, `SUCCEEDED`, `FAILED`, `REVERSED`, `DISPUTED`, and `IN_DOUBT` - are separate records. They do not change the intent's principal status and do not represent real settlement.

There is no persisted `PROPOSED`, `PAID`, or `SETTLED` state. Expiry is evaluated lazily when relevant service methods run.

5.1 Deterministic policy example

The default configuration permits at most INR 1,000 per proposal and INR 5,000 of policy-allowed daily exposure per agent. It requires two distinct demo actor identifiers above INR 500, supports optional payee allowlists and blocklists, and uses a 30-minute default lifetime. These values are demonstration defaults, not risk recommendations.

Daily exposure uses a UTC calendar day and counts policy-allowed proposals in `AWAITING_APPROVAL` and `AUTHORIZED_NO_DISPATCH`. The policy is intentionally simple and cannot be described as fraud detection.

5.2 Idempotency

The database enforces uniqueness for `(agent_id, idempotency_key)`. Repeating a normalized request with the same key returns the prior proposal; changing the request while reusing the key yields a conflict. This protects one database against a common duplicate-submission pattern. It does not establish exactly-once delivery across services or a real rail.

6. Evidence and integrity mechanisms

Parimit uses three different mechanisms. They should not be conflated.

6.1 Proposal digest

A project-specific canonicalizer recursively sorts object keys before hashing a versioned object with SHA-256. For new `parimit-payment-intent-v2` rows, the digest binds the proposal identifier, digest version, idempotency key, requester and optional on-behalf-of label, amount, currency, payee reference, purpose, timestamps, initial status, policy allow/deny result, policy reasons, rules version, and required approval count.

The canonicalizer is not RFC 8785 JSON Canonicalization Scheme. The digest intentionally does not change as the workflow advances, so mutable current status and later approval records are not members of that digest. They are checked separately against HMAC-authenticated decision rows and the hash-linked event history. The implementation also reads pre-v2 databases: migrated v1 rows retain an explicit v1 marker and their original narrower digest rather than having their historical hash silently rewritten. Those v1 rows are archival: review, cancellation, and observation mutations are rejected, and any v1 `AUTHORIZED_NO_DISPATCH` state fails integrity as untrusted. Neither version provides a tenant binding, monotonic state version, one-time consumption nonce, or externally verifiable signature.

6.2 Decision record authentication

Each recorded decision has an HMAC-SHA-256 over the receipt schema version, intent identifier and digest, actor identifier and role, decision, and timestamp. HMAC verification can detect selected changes when the secret remains protected. Because HMAC uses a shared secret, it is not a digital signature, does not provide public verification, and does not prove that a real human controlled the actor identifier. The aggregate receipt returned for an authorized intent is reconstructed from stored rows rather than separately signed as a whole.

6.3 Audit linking

Audit events include the prior event hash and a SHA-256 hash of the canonical event data. Verification can detect selected edits or broken links in the local per-intent view. SQLite does not enforce append-only storage, and a privileged attacker can rewrite an entire chain. The correct description is a **hash-linked local audit trail**, not an immutable ledger.

6.4 State-consistency verification

The alpha integrity verifier recomputes the versioned proposal digest, verifies decision HMACs and audit links, compares the initial audit policy snapshot with stored policy, matches decisions to review events, derives lifecycle state from events, and checks the approval threshold. It also reconciles ordered mock-observation rows with audit evidence for count, status, provider reference, source, timestamp, retry/no-money flags, and, starting in alpha.1, observation ID. Read and mutation paths fail closed when the combined report is invalid. Adversarial tests cover bound policy fields, forged authorization, deleted approvals, and observation edits, deletion, insertion, and reordering. Legacy observations without IDs remain readable only when all fields, order, and count match.

This is still local consistency evidence, not durable authorization proof. A privileged attacker who can rewrite the database, event chain, and protected secret can manufacture a self-consistent history. The aggregate receipt is reconstructed after verification rather than signed as one independently portable object. A stronger future design would add a complete versioned authorization envelope with tenant, state version, actor set, timestamps, nonce, one-time consumption state, asymmetric verification, and an external audit anchor.

7. Capability surfaces

7.1 MCP tools

The MCP server advertises exactly six proposal-safe tools:

Tool	Purpose	Authority deliberately absent
create_payment_proposal	Validate, evaluate, and persist an intent	No credential, signature, or dispatch
get_payment_status	Read proposal and mock observation status	No settlement assertion
cancel_payment_proposal	Cancel an eligible proposal	No refund or rail cancellation
get_policy_decision	Inspect deterministic policy output	No policy mutation
simulate_payment	Create a clearly labelled mock outcome	No network access or money movement
get_payment_audit	Read local hash-linked events	No immutable or external proof

No MCP tool can approve, authorize, execute, initiate, pay, send, transfer, settle, or retry.

7.2 REST routes

REST supports safety metadata, proposal creation/list/read/cancel, policy simulation, demo decisions, audit retrieval and verification, and mock observations. The approval route is kept out of MCP but is not strongly protected: it trusts local role and actor headers. Read/list/audit and mock-observation routes do not implement production access control. The HTTP server limits JSON bodies and applies basic static-content security headers, but it has no TLS termination, CSRF control, rate limiter, or hardened session management.

7.3 Capability statement

The codebase can accurately claim that the shipped MCP registry and default repository contain no live payment execution path. It cannot claim that an adversarial network agent is unable to call the demo HTTP approval endpoint. Real separation requires authenticated identities, route-level authorization, tenant scoping, network segmentation, and independent security testing.

8. Relationship to 2026 agent-payment protocols

Parimit is not an alternative settlement protocol. It occupies an earlier governance position: before a system creates a credential, signed payment payload, checkout completion request, or rail instruction.

System	Primary role	Execution relationship	Where Parimit could sit	Implemented in Parimit now?
NPCI AiNxt OS	General governed runtime for agents and tools, with mandatory gates and an India-centric settlement perimeter	Can govern executable tool actions; its public repository is not a UPI SDK or payment rail	A narrow proposal service used by, or alongside, a governed runtime	No native integration; design influence only
AP2 v0.2	Checkout and Payment Mandates, receipts, roles, and deterministic verification for agent-performed payments	Mandates participate in a journey that obtains credentials and reaches a processor	Before a trusted surface or adapter creates an executable mandate	No
x402 v2	Payment requirements, signed payloads, facilitator verification, and settlement for paid resources	The core flow reaches verification and blockchain settlement; client budget management is out of scope	A client-side budget and review gate before signing a payment payload	No
ACP beta, 2026-04-17 stable spec	Commerce discovery, checkout, orders, authentication, payment handlers, and MCP	Merchant checkout completion can lead to purchase on merchant rails	Before checkout completion, binding approval to an exact cart/session digest	No
UPI and provider products	Regulated payment initiation and settlement through approved participants and products	Moves or reserves actual funds	Only through a separately reviewed executor operated by an authorized participant	No

8.1 NPCI AiNxt OS

NPCI's public AiNxt OS repository describes a broad enterprise AI runtime with mandatory authentication, authorization, compliance, and audit gates. As accessed in September 2026, it also documents a settlement perimeter that reserves India-centric rail destinations, while listing MCP, the TypeScript and Python SDKs, and several other areas as design-only or placeholders. That makes AiNxt important context, but not a public UPI payment SDK.

The chronology also matters. The primary NPCI conference release located for the named new-generation payment initiatives is from GFF 2025, where NPCI described UPI HELP, IoT Payments with UPI, Banking Connect, and UPI Reserve Pay. The source review did not locate an official GFF 2026 release of a public UPI agent SDK or payment protocol. This paper therefore treats the September 2026 AiNxt repository as current technical context, not as evidence of a GFF 2026 UPI-agent API.

Parimit is independently written and narrower. It borrows the architectural lesson that a governance core should not silently become a settlement client. It is not built on AiNxt, has no AiNxt adapter today, is not affiliated with NPCI, and must not use NPCI or UPI marks.

8.2 AP2

AP2 v0.2 defines Checkout and Payment Mandates, signed receipts, deterministic verification responsibilities, a non-agentic Trusted Surface, and both human-present and human-not-present flows. Its artifacts are designed to support an executable commerce journey. Parimit's current receipt is deliberately weaker and non-dispatchable. A future adapter could map a fully verified Parimit authorization object into inputs for an AP2 trusted component, but Parimit must not mint an AP2 payment credential or call a processor inside its core.

8.3 x402

x402 v2 standardizes internet-native payment requirements, payloads, facilitator verification, and settlement across transports including HTTP, MCP, and A2A. The specification explicitly leaves client-side budget management and session handling outside its core. That is a natural integration seam: Parimit could evaluate a parsed payment demand before a separate wallet signs it. The Parimit core must never create the signed payment authorization or call `/settle`.

8.4 ACP

ACP is a beta commerce protocol maintained by OpenAI and Stripe. Its April 2026 stable specification includes cart, feed, order, authentication, and MCP capabilities. The merchant remains the system of record and checkout completion can create an order on existing commerce and payment infrastructure. Parimit could gate that completion by binding review to an exact cart and session digest, but it must not invoke completion or process credentials.

9. Threat model and residual risk

The primary adversary is an untrusted or compromised agent attempting to gain more authority than its declared proposal tools provide. Other threats include injected tool arguments, replay, self-approval, fake dual control, proposal substitution, race conditions, database editing, credential leakage, and ambiguous outcomes.

9.1 Controls demonstrated in the alpha

- A narrow MCP registry with no approval or execution verbs.
- Deterministic validation and policy checks.
- Agent-scoped idempotency in one SQLite database.
- Distinct demo actor identifiers and requester/approver identifier separation in application logic.
- V2 digest binding for immutable proposal fields, initial status, policy result, policy reasons, rules version, and approval threshold.
- State reconstruction and approval-threshold consistency checks before a receipt is returned.
- HMAC authentication of individual decision records.
- Hash-linked audit events.
- Atomic rejection of every later mock observation after `IN_DOUBT`.
- No third-party runtime dependencies and no provider connector.
- Refusal to start when configured as a non-demo deployment.

9.2 Material residual risks

- **Spoofable identity:** headers and actor identifiers are not proof of people, roles, or sessions.
- **Local integrity trust:** mutable state and the approval set are checked across HMAC records and a local event chain, but they are not one externally signed authorization object; legacy v1 rows retain their narrower historical digest and are restricted to non-authorizing archival reads.
- **Concurrency:** transitions do not use a complete expected-version compare-and-set design across processes.
- **Local audit rewrite:** no external anchor prevents a privileged full-history rewrite.
- **Authorization and privacy:** no tenant isolation or robust route-level access control exists.
- **Key security:** the default HMAC secret is development-only and no key manager or rotation system ships.
- **Operational security:** no TLS, CSRF defense, production rate limits, backups, migrations, disaster recovery, or hardened deployment guide.
- **Scanner limits:** the boundary scanner is heuristic lint, not data-flow analysis or formal proof.

These risks are reasons to keep v0.1 local and non-financial, not reasons to attach it quietly to a live API.

10. Verification evidence

The repository contains seventeen automated tests: thirteen service tests and four HTTP/MCP/configuration tests. They cover input validation, scoped idempotency, configured policy limits and lists, one- and two-actor decisions, rejection, lazy expiry, v1 database migration and archival enforcement, a reconstructed v1 threshold-lowering attack, v2 policy/digest binding, forged authorization state, deleted approvals, decision/audit/observation tampering, observation deletion/insertion/reordering, strict approval and observation request bodies, fail-closed post-IN_DOUBT observation handling, the exact MCP tool list, absence of known unsafe HTTP paths, and refusal to start outside demo mode. The boundary scanner examines seven runtime source files for prohibited MCP capability names and common network imports or calls in the core.

On the maintainer's local Node.js 24 environment, all seventeen tests and the boundary scan pass. This paper records local evidence; hosted continuous-integration results are dynamic repository evidence and should be checked independently. Tests still do not cover multi-process races, authenticated route authorization, canonicalization fuzzing, a privileged full-history rewrite of v2 data with key access, or startup verification of every stored row.

The evaluation therefore supports only a bounded conclusion: the checked source and tests demonstrate the intended proposal-only shape under ordinary local execution. They do not prove production security.

11. Hardening roadmap

Phase 1 - complete the integrity model

- 1 Extend the v2 digest and local consistency report into a portable authorization envelope that binds tenant, state version, actor set, creation/expiry, a one-time nonce, and consumption state.
- 2 Make every state change an atomic, conditional transition with an expected version and add multi-process race tests.
- 3 Design a separately authorized reconciliation event with authoritative evidence before permitting any future resolution of a frozen IN_DOUBT state.
- 4 Add adversarial coverage for replay, canonicalization edge cases, startup verification, and privileged full-history rewrite assumptions.
- 5 Define an external anchoring and asymmetric-verification design before any receipt is accepted outside the local demo boundary.

Phase 2 - real identity and durable evidence

- 1 Replace demo headers with OIDC-based sessions and phishing-resistant WebAuthn/passkey confirmation for privileged decisions.
- 2 Introduce tenant and resource authorization for every route.
- 3 Replace shared HMAC decision evidence with versioned public-key signatures or JWS where public verification and key separation are required.
- 4 Adopt a standard canonical form such as RFC 8785 after interoperability analysis.
- 5 Add protected key storage, rotation, revocation, migrations, backups, and externally anchored append-only audit evidence.

Phase 3 - adapters outside the core

Create separately packaged, separately reviewed adapters for AP2, x402, ACP, or regulated provider systems. An adapter may consume a one-time authorization object only after revalidation. It must own provider credentials, network access, replay protection, reconciliation, and settlement semantics. The proposal core must remain unable to import the adapter or reach its network.

No UPI adapter should be presented as usable without a participating bank/PSP relationship, applicable NPCI specifications and approvals, regulatory review, security assessment, and end-to-end certification in the relevant environment.

12. Design invariants for future releases

The following invariants are the intended long-term contract. Items not yet fully enforced are marked as targets.

Invariant	v0.1 status
Agent-facing MCP has no approve, sign, execute, send, settle, or retry tool	Enforced by registry, tests, and heuristic scan
Default repository contains no live payment connector	Implemented
Money uses integer minor units	Implemented for INR
Immutable proposal fields and initial policy decision change the v2 intent digest	Implemented for new v2 rows; legacy v1 is identified and archival-only
Mutable state and approval set are checked against authenticated records and event history	Implemented as local consistency verification
State version, tenant, nonce, consumption state, and approval set are bound in one portable authorization object	Target; not implemented
A receipt is returned only when local state and valid approvals are mutually consistent	Implemented in service read paths; not externally verifiable
Human decisions are tied to strongly authenticated identities	Target; demo identifiers only
IN_DOUBT blocks all later mock observations	Implemented; no reconciliation mechanism in alpha
Audit evidence is externally anchored and independently verifiable	Target; local hash links only
Live execution remains outside the proposal core	Architectural requirement

13. Conclusion

Agent payments need more than a new way to transmit payment data. They need an explicit authority boundary that can tell an agent "you may describe this action, but you cannot approve or execute it."

Parimit v0.1-alpha makes that boundary concrete in a small, inspectable prototype. Its strongest property is negative: the shipped default has no live payment connector and its MCP surface provides no approval or execution capability. Its deterministic policy, v2 initial-decision digest, demo decisions, HMAC records, state-consistency checks, and hash-linked events show how governance evidence can be organized around that boundary.

The same honesty must apply to what is unfinished. Identity is spoofable, integrity evidence remains local rather than independently verifiable, concurrency is not production-safe, reconciliation is not implemented, and the audit log is locally rewritable. The next release should close those gaps before adding any protocol adapter. Execution should be the last component introduced, in a separate trust domain, after the proposal boundary can defend its own invariants.

References

- 1 NPCI, **AiNxt OS**, official repository, accessed September 19, 2026: <https://github.com/npci/ainxt-os>
- 2 NPCI, **RBI Governor unveils New Generation of Digital Payment Initiatives at GFF 2025**: https://www.npci.org.in/uploads/RBI_Governor_unveils_New_Generation_of_Digital_Payment_Initiatives_at_GFF_6494fabdfc.pdf
- 3 NPCI, **UPI Reserve Pay, UPI Operating Circular OC-228**, October 8, 2025: https://www.npci.org.in/uploads/UPI_OC_No_228_FY_2025_26_Enhancement_in_UPI_Single_Block_Multiple_Debits_UPI_Reserve_Pay_a9095c181d.pdf
- 4 Google, **Agentic Payment Protocol v0.2 specification**: <https://github.com/google-agentic-commerce/AP2/blob/main/docs/ap2/specification.md>
- 5 Google, **Agent Payments Protocol is evolving with new capabilities and moving to the FIDO Alliance**, April 28, 2026: <https://blog.google/products-and-platforms/platforms/google-pay/agent-payments-protocol-fido-alliance/>
- 6 x402 Foundation, **x402 Protocol Specification v2**: <https://github.com/x402-foundation/x402/blob/main/specs/x402-specification-v2.md>

- 7 Linux Foundation, **Operational launch of the x402 Foundation**, July 14, 2026: <https://www.linuxfoundation.org/press/linux-foundation-announces-operational-launch-of-x402-foundation-to-standardize-internet-native-payments-for-ai-agents-and-applications>
- 8 OpenAI and Stripe, **Agentic Commerce Protocol**, official repository and 2026-04-17 specification: <https://github.com/agentic-commerce-protocol/agentic-commerce-protocol>
- 9 OpenAI, **Buy it in ChatGPT**, September 29, 2025: <https://openai.com/index/buy-it-in-chatgpt/>
- 10 W3C, **Web Authentication: An API for accessing Public Key Credentials - Level 3**, Recommendation, August 25, 2026: <https://www.w3.org/TR/2026/REC-webauthn-3-20260825/>
- 11 NIST, **Digital Identity Guidelines: Authentication and Authenticator Management, SP 800-63B-4**, July 2025: <https://csrc.nist.gov/pubs/sp/800/63/b/4/final>
- 12 RFC 8785, **JSON Canonicalization Scheme**, June 2020: <https://www.rfc-editor.org/info/rfc8785/>
- 13 RFC 7515, **JSON Web Signature**, May 2015: <https://www.rfc-editor.org/info/rfc7515/>
- 14 RFC 9162, **Certificate Transparency Version 2.0**, December 2021: <https://www.rfc-editor.org/rfc/rfc9162.html>

Appendix A - Reproducibility snapshot

- Release label: 0.1.0-alpha.1
- Runtime: Node.js 24
- Language: TypeScript
- Persistence: built-in SQLite
- Runtime package dependencies: none
- Persisted tables: intents, approvals, observations, audit events
- Automated tests in repository: 17
- Runtime files checked by boundary scanner: 7
- MCP protocol identifier advertised by server: 2025-06-18
- Default deployment: local demo only
- License: MIT

Appendix B - Independent project statement

Parimit is independently implemented. Public protocol specifications and the NPCI AiNxt OS repository were studied to understand the surrounding ecosystem and to distinguish governance from execution. No claim is made that Parimit is sponsored, reviewed, certified, or endorsed by NPCI, Google, FIDO Alliance, x402 Foundation, Linux Foundation, OpenAI, Stripe, any payment provider, or any regulator. Names and marks belong to their respective owners.